

Pipeline del *Machine Learning*

Dal problema al modello valutato — formulazione, dati, scelta del modello, training e valutazione.

Dante Shintaro Cometti

Intro to ML / AI · Part 3 of 3



3.1 *Definire il problema*

02

— SEZIONE DUE —

APERTURA · PART 3

Formulazione del *problema*.

Bisogna fare attenzione a problemi formulati male: *etichetta sbagliata* , orizzonte temporale sbagliato, *metrica sbagliata* , data leakage, ottimizzazione sbagliata — anche se l'addestramento del modello è «corretto».

Formulazione del *problema*.

La **formulazione del problema** è il passaggio in cui si traduce un obiettivo del mondo reale in un task di ML preciso, con **input**, **output**, vincoli e valutazione chiaramente definiti.

Cosa stai predicendo?

Definisci il tuo output chiaramente, è un valore continuo (regressione), una categoria (classificazione), o una struttura (clustering, ranking)?

Quali sono i tuoi dati di input? Identifica le tue feature, quali informazioni sono disponibili al momento dell'inferenza che il modello può effettivamente utilizzare?

Qual è la tua label? Nel

apprendimento

supervisionato, qual è la

verità di riferimento su cui stai addestrando?

Come appare il successo?

Definisci una metrica prima di iniziare

Quali sono i tuoi vincoli? Latenza, memoria, disponibilità dei dati, requisiti di interpretabilità, questi spesso determinano quale approccio è fattibile

Una nota sul mondo reale.

I modelli ben allenati, testate e validati, verranno (si spera) implementati nel mondo reale.



3.2 *Dati*

02

— SEZIONE DUE —

*Prima del modello vengono i dati la
qualità del modello è la qualità di ciò che
gli diamo in pasto.*

Dati.

Prima di scegliere un modello, vale la pena fermarsi e fare domande sui dati a disposizione.

⁰¹ Puoi **visualizzare** i dati per comprenderli meglio?

⁰² So quali **classi** ho?

⁰³ Quali dati ho?

⁰⁴ Che tipo di dati ho?

⁰⁵ Quanto è *abbastanza* dati?

⁰⁶ Qual è l'**obiettivo**?

⁰⁷ Ho abbastanza dati?

⁰⁸ Quali sono i tuoi **input**?

Feature Crafting e *Analisi*.

Una **feature** è una variabile di input presa o derivata dai tuoi dati.

01 · COSA

Feature

ciò che fornisci al modello

Ogni variabile di input che il modello vede in fase di training e di predizione.

02 · COME

Feature crafting

come crei o migliori quegli input

Costruzione, trasformazione e selezione delle variabili che entreranno nel modello.

03 · VERIFICA

Feature analysis

controlli sugli input

Verifichi che gli input siano *informativi, stabili e appropriati* per il task.

Feature *Crafting*.

Il processo di **creare**, **trasformare** e **selezionare** feature per far apprendere meglio il modello.

Pulizia e preprocessing

Gestire valori mancanti — serve una categoria «sconosciuto»? Trattare outlier, standardizzare o normalizzare variabili numeriche, codificare variabili categoriche, trasformare il testo in dati.

Trasformazioni

A volte serve trasformare variabili, ad esempio `log` per il reddito, raggruppare o discretizzare variabili come età o tempo, oppure combinare variabili.

Aggregazione

Conteggi o medie: «*spesa media negli ultimi 30 giorni*», «*numero di pagamenti in ritardo negli ultimi 12 mesi*» .

Feature *Analysis*.

Controllare se le feature sono **utili**, **valide**, **stabili** e sicure da usare.

- 1** **Utilità predittiva**
Correlazione per la regressione, *mutual information*, feature importance nei modelli ad albero, *permutation importance*, test di ablazione
- 2** **Relazione con il target**
Grafici delle distribuzioni per classe, *partial dependence*, riepiloghi SHAP, controlli di monotonicità: *il rischio aumenta quando aumenta il*
- 3** **Controlli di data leakage**
Assicurarsi che una feature non sarebbe nota al momento della predizione
- 4** **Stabilità nel tempo · drift**
La distribuzione della feature cambia — ad esempio prima e dopo un cambio di policy?
- 5** **Fairness e compliance**
La feature è un *proxy* per attributi protetti, ad esempio il codice postale come proxy per etnia o reddito

Problemi nei *dati*.

Domande principali da considerare prima di andare avanti:

- 1** Le **etichette** sono affidabili e definite in modo coerente?
- 2** Il dataset è **rappresentativo** di chi o cosa verrà predetto?
- 3** Ci sono segnali di **drift** tra dati di training e dati di test?

Che cosa può andare storto con i dati · *I*

- 1** Problemi di qualità dei dati
Valori mancanti, outlier o valori estremi, rumore ed errore di misurazione, duplicati, formati o unità incoerenti, valori invalidi
- 2** Problemi di etichetta · target
Etichette scorrette, etichette ambigue, cambiamento della definizione dell'etichetta nel tempo, sbilanciamento delle classi.
- 3** Data leakage
Target leakage: una feature contiene direttamente o indirettamente la risposta. **Contaminazione train-test**: duplicati o quasi duplicati tra split. **Time leakage** : uso di dati futuri nelle feature per predizioni passate.
- 4** Problemi di integrazione dei dati e pipeline
Join scorretti, many-to-many, join impropri nel tempo. Problemi di *entity resolution* — la stessa persona appare con più ID.

Che cosa può andare storto con i dati · II



- 5 Problemi di campionamento e rappresentatività
Bias di campionamento, dati di training non rappresentativi dell'uso reale; **selection bias** — osservi gli esiti solo per un sottoinsieme (es. solo prestiti approvati)
- 6 Problemi con le feature
Categoriche ad alta cardinalità (rischio di sparsità/overfit)
- 7 Dimensione del dataset e dimensionalità
Tropo pochi dati, soprattutto per modelli complessi; alta varianza e overfitting; troppe feature rispetto ai campioni; pochi esempi positivi per eventi rari; gruppi sbilanciati con sottopopolazioni troppo piccole.
- 8 Drift e mismatch con i dati di produzione
Training-serving skew — feature calcolate diversamente in produzione rispetto al training; **concept drift**, la relazione tra input e target cambia (es. le tattiche di frode evolvono); **covariate shift**, cambia la distribuzione degli input (nuove demografie di utenti); cambiamenti di strumentazione, logging o raccolta che rompono il significato delle feature.

3.25 *Aneddoto al tempo tragico, ora comico, spero*

— SEZIONE DUE —

Feature analysis...

Data una linea di log in linguaggio naturale decider se è **ANOMALA** o **NO**

Training

INPUT: Migliaia di Log files

OUTPUT: Modello che mappa le relazioni
logs e **ANOMALIA/NORMALE**

Inference

INPUT: Linea di log

OUTPUT: **ANOMALIA/NORMALE**

Feature analysis...

Data una linea di log in linguaggio naturale decider se è **ANOMALA** o **NO**

[2026-05-21 05:16:44] ERROR — Power amplifier module overheating (91°C, limit is 85°C). TX power automatically reduced to 40 dBm. Fan ramped to 100%.

[2026-05-21 05:17:30] PA temperature coming down, now at 87°C. Still above threshold but trending stable. Thermal throttle remains active.

[2026-05-21 05:20:00] Routine link quality report: 20 minutes uptime, 487 frames transmitted OK, 451 received OK, 7 total errors. Bit error rate estimated at 1.2e-5 — within acceptable range.

[2026-05-21 05:24:11] Incoming frame from NODE-9. Clean decode, no issues. Heartbeat acknowledged.

[2026-05-21 05:31:05] ERROR — Lost contact with NODE-7 entirely. No response to three consecutive polls. Flagged as unreachable, operator alert queued.

[2026-05-21 05:45:00] All other nodes nominal. Uplink to BRAVO-2 stable. PA temperature back to 82°C, throttle lifted, TX power restored to 47 dBm.

Quale è stato il mio errore?

Quale è stato l'errore degli Svedesi?

3.3 *Modello*

Scegliere il tipo di ragionamento — classificazione, regressione o clustering — e mapparlo sul workflow giusto.

03

— SEZIONE TRE —

Quale modello *scelgo*?

0 1

Classificazione

spam vs non spam

0 2

Regressione

predire il prezzo di una casa

0 3

Clustering / descrittivo

trovare gruppi



Adattarsi al *workflow*.

FEATURE	SUPERVISIONATO	NON SUPERVISIONATO	RINFORZO
Requisito dei dati	Dati etichettati	Dati non etichettati	Ambiente interattivo
Processo di apprendimento	Impara da coppie input-output	Scopre pattern nei dati	Impara da azioni e ricompense
Tipi di modello	Classificazione, regressione	Clustering, riduzione della dimensionalità	Q-learning, policy gradients
Obiettivo	Predire o classificare nuovi punti dati	Identificare strutture nascoste	Massimizzare ricompense cumulative
Esempi	Rilevamento spam, classificazione immagini	Segmentazione clienti, rilevamento anomalie	Giochi, robotica
Valutazione	Accuracy, precision, recall, MSE	Silhouette score, explained variance	Ricompensa cumulativa
Sfide	Richiede grandi dataset etichettati, overfitting	Interpretabilità, difficoltà di valutazione	Trade-off esplorazione-sfruttamento, efficienza campionaria

3.4 Training: configurazione del *modello*

— SEZIONE QUATTRO —

Dividere i dati, scegliere gli iperparametri, valutare in modo onesto.

Testing e *Training*.

Dividi i dati → **train / test**.

Split casuale — ad esempio **80% train / 20% test**.

Usa uno **split stratificato** per la classificazione, per mantenere proporzioni di classe simili in entrambi i set, importante con classi sbilanciate.

Se i dati sono ordinati temporalmente, usa uno **split basato sul tempo** invece di uno casuale.

Potresti volere un **validation set** : cross-validation + tuning + selezione delle soglie.

Riaddestra la configurazione migliore sull'**intero train**.

Valuta **una sola volta** sul test.

SPLITTING · ESEMPIO 80/20

TRAIN · 80%

TEST · 20%

CON VALIDATION SET

TRAIN · 60%

VAL · 20%

TEST · 20%

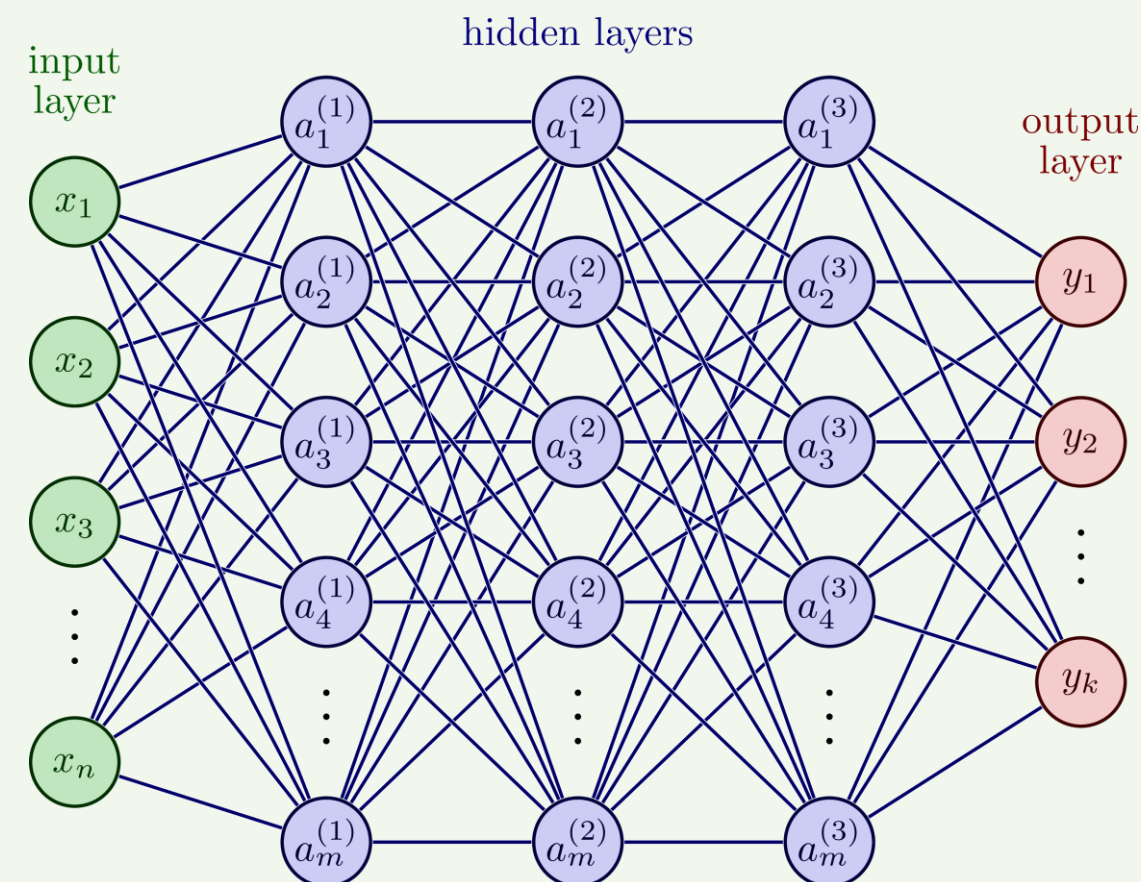
*Il test set va toccato **una sola volta**, alla fine — per misurare l'errore reale.*

Tuning degli *iperparametri*.

Ogni modello ha impostazioni specifiche che puoi specificare **prima dell'addestramento**: questi sono gli **iperparametri**.

Possono essere **modificati** e **testati** sui dati per assicurarsi che il modello funzioni bene su dati mai visti.

Provare varie impostazioni e misurare le prestazioni sui **dati di validazione** assicura che siano impostate correttamente.



Learning rate: quanto grande è ogni passo di aggiornamento durante l'allenamento. Immagina di scendere una montagna al buio — passi troppo grandi e salti la valle, passi troppo piccoli e non arrivi mai

Numero di layer: quanti strati ha la rete. Una rete con 2 layer è come un ragionamento semplice, una con 20 layer è come un ragionamento molto più profondo e astratto

Neuroni per layer: quanti neuroni ci sono in ogni strato. Più neuroni = più capacità di imparare pattern complessi, ma anche più rischio di overfitting

Batch size: quanti esempi vede il modello prima di aggiornare i pesi. Un batch piccolo (es. 32) aggiorna spesso ma con rumore, un batch grande (es. 512) aggiorna meno spesso ma in modo più stabile

Numero di epoche: quante volte il modello vede l'intero dataset durante l'allenamento. Troppe epoche → overfitting, troppo poche → underfitting

3.5 *Valutazione*

Come si racconta in modo onesto il risultato di un modello — con una baseline e una metrica chiara.

05

— SEZIONE CINQUE —

Presentare i *risultati*.

I risultati dovrebbero presentare una o più **metriche di valutazione**.

Metriche di valutazione

Quante volte hai ragione: su 100 previsioni, quante erano corrette? Se il modello indovina il beam giusto 80 volte su 100, hai un'accuratezza dell'80%. Semplice ma può ingannare se le classi sono sbilanciate.

Di quanto ti sbagli in media: calcoli la differenza tra il valore previsto e quello reale per ogni esempio, e fai la media. Se prevedi che un utente abbia 30 anni ma ne ha 35, l'errore è 5. Utile per problemi di regressione

Quanto penalizzi gli errori grandi: simile alla metrica precedente ma gli errori grandi vengono penalizzati molto di più di quelli piccoli. Prevedere 30 invece di 31 è accettabile, prevedere 30 invece di 60 è un disastro.

Quanto sei preciso quando dici sì: su tutti i casi in cui il modello ha detto "questo è il beam dominante", quante volte aveva davvero ragione? Utile quando il costo di un falso positivo è alto.

Presentare i *risultati*.

I risultati dovrebbero presentare una **baseline**.

Una baseline semplice viene confrontata con i tuoi risultati per mostrare, auspicabilmente, un *miglioramento* .

Le baseline dipendono dai dati e dal modello usati.

Media: prevedi sempre la media del training set. Se stai prevedendo l'età, prevedi sempre 45 anni. Qualsiasi modello decente deve battere questo.

Valore precedente (Last Known Value): prevedi che il valore al tempo $t+1$ sia uguale al valore al tempo t .

Modello lineare semplice: una regressione lineare o logistica. Pochissimi parametri, facile da interpretare, spesso già competitivo.

Regola euristica: una regola fissa basata sulla conoscenza del dominio. Per esempio "assegna sempre il beam più vicino geograficamente all'utente".

Random: prevedi un valore casuale. Serve solo per avere un lower bound. Se il tuo modello non batte il caso, c'è un problema serio.

Modello più semplice della stessa famiglia: se stai usando una rete neurale profonda con 20 layer, confrontala con una rete con un solo layer nascosto.

Una nota sulla *riproducibilità*.



Qualcuno, incluso *te stesso nel future*, deve poter rieseguire la stessa pipeline e ottenere gli stessi risultati dati gli stessi dati e lo stesso codice.

1 Dati

Dettagli sul **dataset esatto** usato, inclusi eventuali passaggi di pulizia.

2 Feature

Come ogni feature è stata calcolata — **codice + parametri + finestre temporali**.

3 Training del modello

Algoritmo, iperparametri, metodo di training.

4 Valutazione

Esatti **split train/test**, baseline, metriche.

5 Ambiente

Versioni dei pacchetti, differenze hardware.

3.55 *Aneddoto al tempo tragico, ora comico, spero*

— SEZIONE DUE —

.....

Prevedere la richiesta di VM, basandosi su richieste passate così da pre-inizializzare le VM e eseguire test più rapidamente

Training

INPUT: Serie storiche di richieste di VM

OUTPUT: Modello che mappa le relazioni fra orari/giorni e richieste di VM

Inference

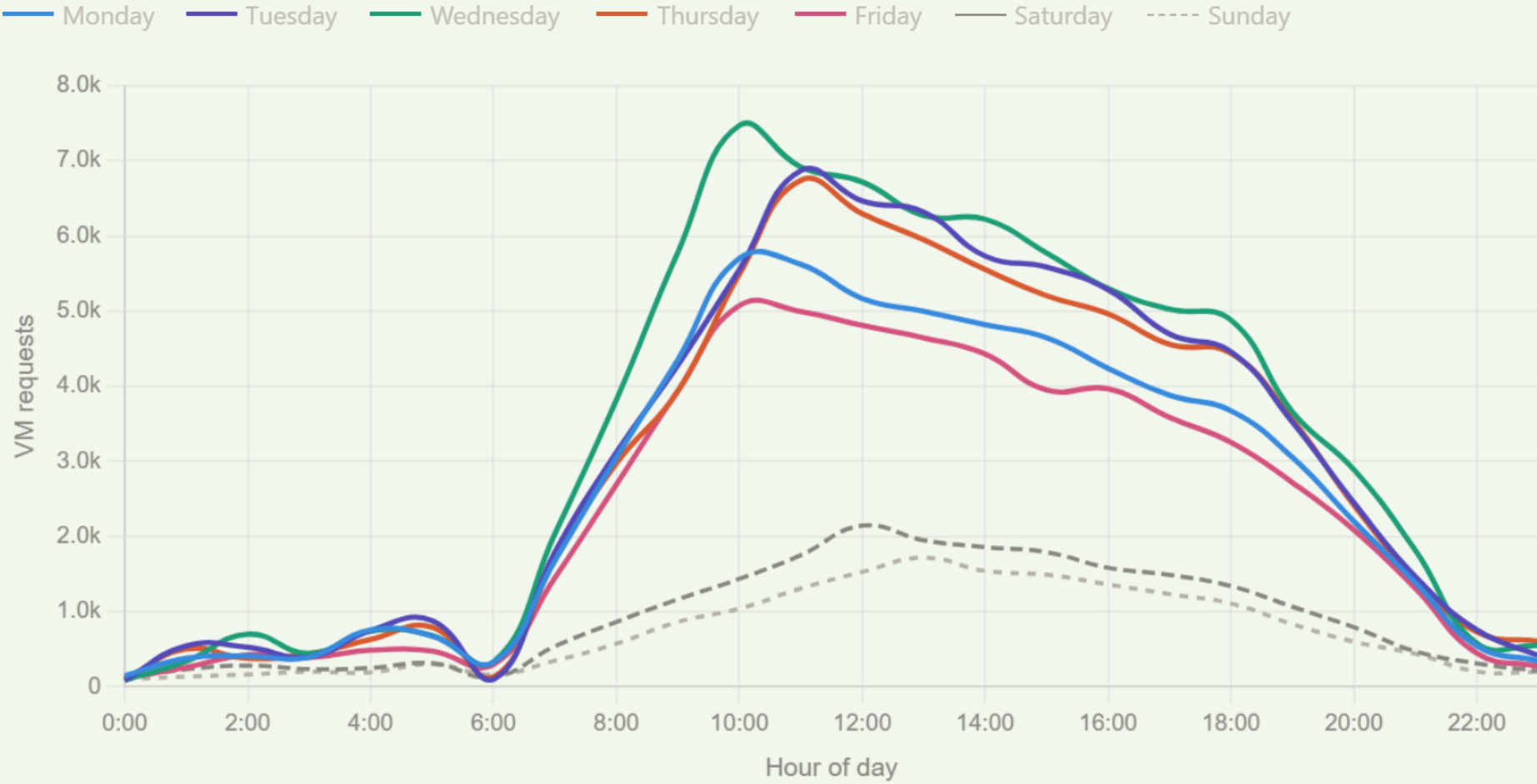
INPUT: Richieste di VM del Giorno prima ($t-1$)

OUTPUT: Stima delle richieste di VM per oggi (t)

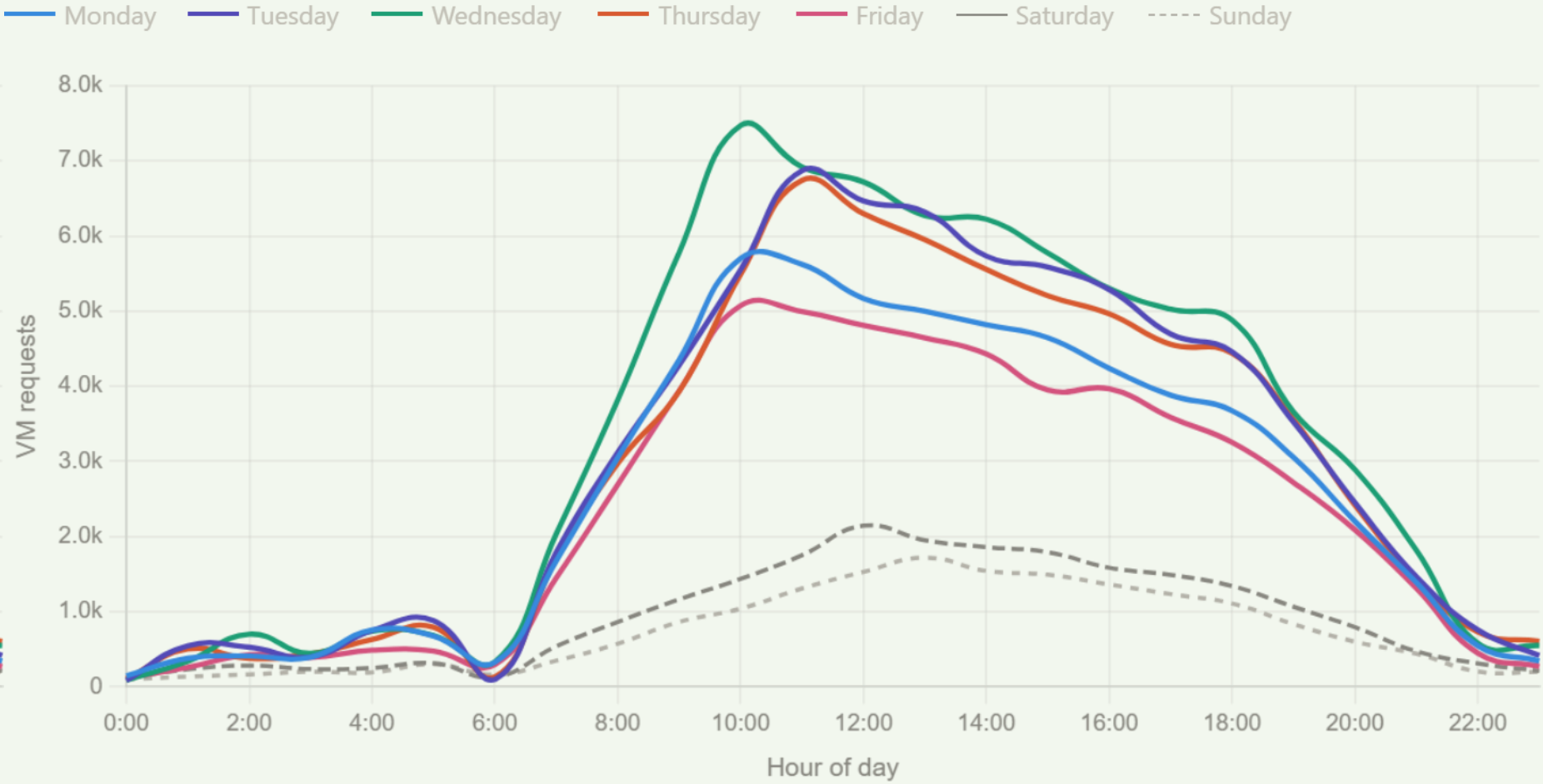
.....

Prevedere la richiesta di VM, basandosi su richieste passate così da pre-inizializzare le VM e eseguire test più rapidamente

Settimana scorsa



Questa settimana



Quale è stato il mio errore?

3.6 *Grazie*

05